Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 6

Disciplina: Tehnici de programare

Tema: ” Informații invizibile: forensics și steganografie ”

A efectuat : st. gr.SI – 264 Șoimaru Gabriel

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Lucrarea A

A1. Text și octeți: încălzirea

Creează arsenal/forensics/decode.py. Fixează-ți întâi diferența dintre text și octeți, de care atârnă tot.

A2. Recunoști stratul după formă

Scrie o funcție care, dat un șir, ghicește ce codare e, uitându-se la caractere. Nu decoda încă, doar clasifică.

A3. Desfaci stratul recunoscut

Acum chiar decodează, în funcție de ce a zis A2. Întoarce și tipul, și rezultatul.

A4. Straturi multiple, într-o buclă

Un rezultat poate fi tot o codare. Decodează în buclă până iese text citibil.

A5. Spargi un XOR cu cheie de un octet

Citește probe/xor.bin (octeți), încearcă toate cele 256 de chei și păstrează rezultatul care arată a text.

A6. Spargi un hash prin dicționar

Din probe/hashuri.txt ia un hash. Identifică algoritmul după lungime, apoi caută parola într-o wordlist.

A7. argparse, fișa JSON și un test

Leagă totul într-o unealtă comandată din afară, salvează rezultatul ca fișă, și scrie un test.

Lucrarea B

B1. Magic bytes: tipul real, nu extensia

Deschide un fișier în mod binar și citește-i primii octeți. Compară-i cu semnăturile cunoscute.

B2. strings: textul citibil dintr-un binar

Scoate secvențele de caractere imprimabile dintr-un fișier, cu un regex pe octeți.

B3. Metadate EXIF și GPS din poză

Citește blocul EXIF din probe/foto.jpg și scoate coordonatele GPS. Convertește-le în grade zecimale.

B4. File carving: scoți fișierul ascuns

probe/ascuns.png are o arhivă ZIP lipită la coadă. Găsește-i semnătura și salveaz-o separat.

B5. Ascunzi un mesaj prin LSB

Scrie un mesaj în ultimii biți ai pixelilor dintr-o poză gazdă. Rezultatul arată identic cu originalul.

B6. Dezvălui mesajul înapoi

Citește ultimii biți ai pixelilor din stego.png, regrupează-i câte 8 și reconstituie mesajul.

EFECTUAREA LUCRĂRII

Lucrarea A

A1

```
s = "Salut"
b = s.encode() # bytes
print(b, b[0]) # b"Salut" 83
# TODO: tipareste b.hex() si refa bytes din hex cu bytes.fromhex(...)
h = b.hex()
print(h)
print(bytes.fromhex(h))
```

Figură 1.1.1 – Codul A1

```
b'Salut' 83
53616c7574
b'Salut'
```

Figură 1.1.2 – Rezultatul A1

A2

```
def ghici_strat(s):
    # TODO: daca are %xx -> "url"
    if re.search(r'%[0-9a-fA-F]{2}', s):
        return "url"
    # TODO: daca doar 0-9 a-f si lungime para -> "hex"
    if re.fullmatch(r'[0-9a-fA-F]+', s) and len(s) % 2 == 0:
        return "hex"
    # TODO: daca doar A-Za-z0-9+/= -> "base64"
    if re.fullmatch(r'[A-Za-z0-9+/=]+', s):
        return "base64"
    return "necunoscut"
print(ghici_strat("%2Fetc")) # url
print(ghici_strat("48656c6c6f")) # hex
print(ghici_strat("U2FsdXQ=")) # base64
```

Figură 1.2.1 – Codul A2

```
url
hex
base64
```

Figură 1.2.2 – Rezultatul A2

A3

```python
def desfa(s):
    strat = ghici_strat(s)
    # TODO: pentru fiecare strat, decodeaza
    # base64 -> base64.b64decode(s)
    if strat == "base64":
        return strat, base64.b64decode(s)
    # hex -> bytes.fromhex(s)
    if strat == "hex":
        return strat, bytes.fromhex(s)
    # url -> unquote(s).encode()
    if strat == "url":
        return strat, unquote(s).encode()
    # intoarce (strat, rezultat_bytes)
    return strat, s.encode()
print(desfa("%2Fetc")) # url
print(desfa("48656c6c6f")) # hex
print(desfa("U2FsdXQ=")) # base64
```

Figură 1.3.1 – Codul A3

```
('url', b'/etc')
('hex', b'Hello')
('base64', b'Salut')
```

Figură 1.3.2 – Rezultatul A3

A4

```
val = open("probe/mesaj.txt").read().strip()
for _ in range(8):
    strat, b = desfa(val)
    # TODO: daca citibil(b): opreste si tipareste
    # altfel val = b.decode(); continua
    if citibil(b) and ghici_strat(b.decode()) == "necunoscut":
        print(b)
        break
    else:
        val = b.decode()
```

Figură 1.4.1 – Codul A4

```
b'FLAG{straturi_de_encoding}'
```

Figură 1.4.2 – Rezultatul A4

A5

```
date = open("probe/xor.bin", "rb").read()
for k in range(256):
    clar = bytes(b ^ k for b in date)
    # TODO: daca citibil(clar):
    # print("cheie:", k, clar.decode())
    if citibil(clar):
        print("cheie:", k, clar.decode())
        break
```

Figură 1.5.1 – Codul A5

```
cheie: 128 Un singur octet tine tot secretul... gaseste-l! FLAG{xor_un_octet}
```

Figură 1.5.2 – Rezultatul A5

A6

```
tinta = open("probe/hashuri.txt").readline().strip()
print(len(tinta)) # 32 md5 / 40 sha1 / 64 sha256
for cuv in open("probe/wordlist.txt", encoding="latin1"):
    cuv = cuv.strip()
# TODO: daca hashlib.md5(cuv.encode()).hexdigest()
# == tinta: print("gasit:", cuv); break
    if hashlib.md5(cuv.encode()).hexdigest() == tinta:
        print("gasit:", cuv)
        break
```

Figură 1.6.1 – Codul A6

```
32
gasit: password
```

Figură 1.6.2 – Rezultatul A6

A7

```
for _ in range(8):
    strat, b = desfa(val)
    if citibil(b) and ghici_strat(b.decode()) == "necunoscut":
        print(b)
        break
    else:
        val = b.decode()
rezultat = b.decode()
date = {
    "strat": strat,
    "rezultat": rezultat
}
with open("arsenal/forensics/fisa.json", "w") as f:
    json.dump(date, f)
```

Figură 1.7.1 – Codul A7

```
{"strat": "hex", "rezultat": "FLAG{straturi_de_encoding}"}
```

Figură 1.7.2 – Rezultatul A7 (Conținutul din fișa.json)

```python
from arsenal.forensics.decode import desfa
def test_base64():
    strat, b = desfa("U2FsdXQ=")
    assert strat == "base64"
    assert b == b"Salut"
```

Figură 1.7.3 – Codul din test_decode.py

```
(.venv) PS C:\Users\Gabriel\Desktop\Gabi\Uni\Tehnici de programare\LabTP6> python -m pytest teste -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\Gabriel\Desktop\Gabi\Uni\Tehnici de programare\LabTP6\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Gabriel\Desktop\Gabi\Uni\Tehnici de programare\LabTP6
collected 1 item

teste/test_decode.py::test_base64 PASSED
```

Figură 1.7.4 – Rezultatul testului test_decode.py

Lucrarea B

B1

```python
def tip_real(cale):
    cap = open(cale, "rb").read(8)
    # TODO: daca cap incepe cu b"%PDF" -> "pdf"
    # b"\x89PNG" -> "png" ; b"\xff\xd8" -> "jpeg"
    # b"PK\x03\x04" -> "zip/docx"
    if cap.startswith(b"%PDF"):
        return "pdf"
    if cap.startswith(b"\x89PNG"):
        return "png"
    if cap.startswith(b"\xff\xd8"):
        return "jpeg"
    if cap.startswith(b"PK\x03\x04"):
        return "zip/docx"
    return cap.hex()
print(tip_real("probe/foto.jpg"))
```

Figură 2.1.1 – Codul B1

```
jpeg
```

Figură 2.1.2 – Rezultatul B2

B2

```
import re
date = open("probe/ascuns.png", "rb").read()
# TODO: for m in re.findall(rb"[ -~]{4,}", date):
# print(m.decode())
# cauta un indiciu: o cale, un mesaj, un nume de fisier
for m in re.findall(rb"[ -~]{4,}", date):
    print(m.decode())
```

Figură 2.2.1 – Codul B2

```
IHDR
IDATx
IEND
secret.txtFLAG{ascuns_prin_carving}
secret.txtPK
```

Figură 2.2.2 – Rezultatul B2

B3

```
from PIL import Image
from PIL.ExifTags import TAGS
exif = Image.open("probe/foto.jpg")._getexif() or {}
for id, val in exif.items():
    print(TAGS.get(id, id), "=", val)
# TODO: ia blocul GPS (tag 34853) si tipareste
# latitudinea si longitudinea
gps = exif.get(34853)
g, m, s = gps[2]
lat = g + m / 60 + s / 3600
g, m, s = gps[4]
lon = g + m / 60 + s / 3600
print("lat:", float(lat), "lon:", float(lon))
```

Figură 2.3.1 -Codul B3

```
Model = iPhone 13
DateTime = 2026:09:14 10:30:00
GPSInfo = {1: 'N', 2: (47.0, 1.0, 0.0), 3: 'E', 4: (28.0, 51.0, 27.0)}
Make = Apple
lat: 47.016666666666666 lon: 28.8575
```

Figură 2.3.2 – Rezultatul B3

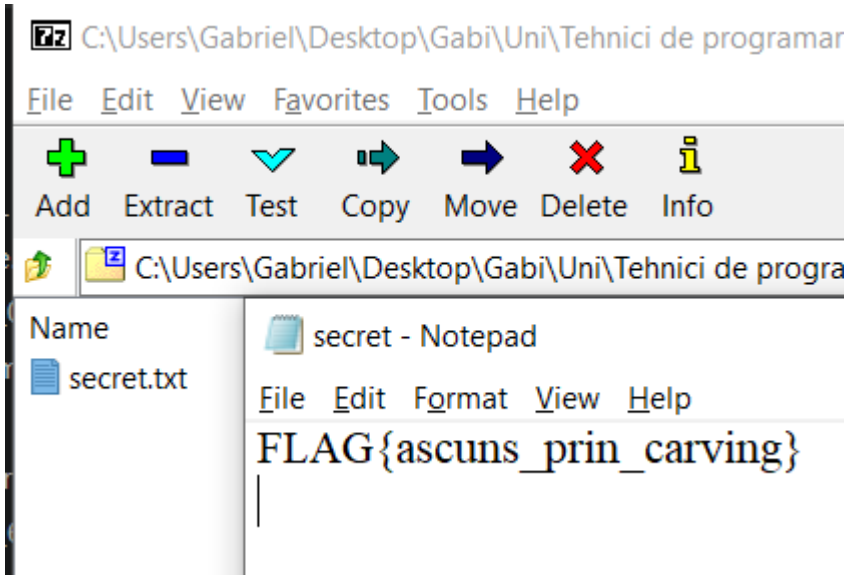
B4

```
date = open("probe/ascuns.png", "rb").read()
i = date.find(b"PK\x03\x04")
# TODO: daca i != -1:
# open("gasit.zip","wb").write(date[i:])
# print("arhiva extrasa de la octetul", i)
# apoi dezarhiveaza si vezi ce era inauntru
if i != -1:
    open("gasit.zip","wb").write(date[i:])
    print("arhiva extrasa de la octetul", i)
```

Figură 2.4.1 -Codul B4

```
arhiva extrasa de la octetul 782
```

Figură 2.4.2 – Rezultatul B4



Figură 2.4.3 – Rezultatul dezarhivării B4

B5

```python
from PIL import Image
img = Image.open("probe/foto.jpg").convert("RGB")
px = list(img.getdata())
mesaj = b"FLAG{ascuns_in_pixeli}"
biti = "".join(f"{x:08b}" for x in mesaj)
# TODO: pentru fiecare bit, ia un canal si pune
# canal = (canal & ~1) | int(bit)
# TODO: img.putdata(px_nou); img.save("stego.png")
canale = [c for pixel in px for c in pixel]
for i, bit in enumerate(biti):
    canale[i] = (canale[i] & ~1) | int(bit)

px_nou = [tuple(canale[i:i+3]) for i in range(0, len(canale), 3)]
img.putdata(px_nou)
img.save("stego.png")
```

Figură 2.5.1 – Codul B5

```
c:\Users\Gabriel\Desktop\Gabi\Uni\Tehnici de programare\LabTP6\arsenal\forensics\stego.py:3: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow
14 (2027-10-15). Use get_flattened_data instead.
  px = list(img.getdata())
```

Figură 2.5.2 -Rezultatul B5



Figură 2.5.3 – stego.png

B6

```
px = list(Image.open("stego.png").convert("RGB").getdata())
biti = ""
for (r, g, b) in px:
# TODO: aduna ultimii biti (r&1, g&1, b&1)
    biti += str(r & 1) + str(g & 1) + str(b & 1)
# TODO: grupeaza cate 8 biti -> octeti -> text
# opreste-te la primul octet 0 (final de mesaj)
mesaj = b""
for i in range(0, len(biti), 8):
    octet = int(biti[i:i+8], 2)
    if octet == 0:
        break
    if not 32 <= octet < 127:
        break
    mesaj += bytes([octet])
print(mesaj.decode())
```

Figură 2.6.1 – Codul B6

```
FLAG{ascuns_in_pixeli}
```

Figură 2.6.2 – Rezultatul B6

CONCLUZIE

În această lucrare am folosit metode de steganografie pentru a ascunde și a găsi unele informații ascunse in diferite metode: codificare și decodificare precum base64 și hex(A2-A4, A7), XOR(A5), hash-uri(A6), tipul real al fișierelor (B1) și textul citibil din binare (B2), metadate EXIF și GPS (B3), fișiere lipite la coada unui PNG (B4) și mesaje ascunse în pixeli prin LSB (B5-B6).